

RT-Thread 移植实录：从空目录到 msh 跑起来

以 ESP32-S31 (RISC-V RV32, bootROM 直启、不用 ESP-IDF 构建系统) 为例

2026-09-23

- [0. 这份文档怎么读](#)
- [1. 前置知识：三块必须先搞懂的东西](#)
 - [1.1 RT-Thread 是什么 \(只讲移植必须知道的\)](#)
 - [1.2 自动初始化表：RT-Thread 的“骨架”，也是移植里最容易踩的坑](#)
 - [1.3 一个“裸机工程”到底缺什么](#)
- [2. 先看清这块板怎么启动 \(它决定了移植方式\)](#)
- [3. 移植步骤 \(九步，每步都有验证方法\)](#)
 - [步骤 1：目录骨架、工具链、拿到 RT-Thread 源码](#)
 - [步骤 2：链接脚本 `bsp/linker.ld`](#)
 - [步骤 3：启动汇编 `bsp/startup.S`](#)
 - [步骤 4：`bsp/rtconfig.h` \(Nano 配置\)](#)
 - [步骤 5：控制台 `bsp/drv\_usj.c` \(能打印，就成功了一半\)](#)
 - [步骤 6：时钟与 tick —— `bsp/drv\_systick.c` + CLIC 路由](#)
 - [步骤 7：陷阱入口与异常现场 —— `bsp/trap\_gcc.S` + `bsp/trap\_handler.c`](#)
 - [步骤 8：构建脚本 `tools/build.ps1` \(编译 → 链接 → 生成镜像 → 烧录\)](#)
 - [步骤 9：让 msh 跑起来 \(最后一步，其实不用写代码\)](#)
- [4. 三个必踩的坑 \(现象 → 根因 → 修法\)](#)
 - [坑 1：所有 `INIT\_\*\_EXPORT` 的函数一个都不执行，但系统“看上去启动了”](#)
 - [坑 2：堆把“表”写花了 → 遍历初始化表时跳到 `0x8c`](#)
 - [坑 3：`INIT\_BOARD\_EXPORT` \(1 级\) 永远不执行](#)
- [5. 步骤 → 文件 → 验证 总表](#)
- [6. 往下走：设备框架 \(你已经熟悉，这里只做衔接\)](#)
- [附录 A：本文用到的关键地址与出处](#)
- [附录 B：术语速查](#)

0. 这份文档怎么读

写给谁： C 语言没问题、会看寄存器手册、**没碰过 RT-Thread** 的人 (比如我自己上手时的状态)。外设驱动 (SPI/I2C/GPIO 那一套) 不在本文范围 —— 本文只讲到 **msh 能敲命令、能回显** 为止。

讲的是一次真实移植： 把 RT-Thread Nano 跑在一块 **ESP32-S31** 开发板上，**完全不使用 ESP-IDF 的构建系统** (不用 `idf.py`、不用 CMake、不用它的二级 bootloader)，自己写启动汇编、链接脚本、编译脚本，镜像由 **芯片自带的 bootROM 直接加载**。

怎么配合工程看： 所有代码都在 `projects/rtt_nano_s31/`，本文每讲一步都会给出**文件名 + 关键代码 + 怎么验证**。建议边看边打开对应文件，这个工程的每个文件头部都写明了“为什么这么写”。

一句话主线 (后面所有细节都挂在这条线上)：

让一块没有操作系统、没有 C 运行环境、没有中断向量的裸芯片，依次长出：能跑 C 的最小环境 → 能打印的串口 → 会走的时钟 (tick) → 能响应中断 → 最后长出 msh。

1. 前置知识：三块必须先搞懂的东西

1.1 RT-Thread 是什么（只讲移植必须知道的）

RT-Thread 是一个国产 RTOS，代码结构像“内核 + 一堆可选组件”。对移植来说，你要知道四件事：

- ① 它是“库 + 骨架”，不是一个能直接烧的固件。内核本身只提供：线程调度、IPC（信号量/互斥量/邮箱/消息队列）、内存管理、定时器、设备框架、shell (msh)。板级初始化、时钟、串口、中断向量，全都得你写——这就是“BSP 移植”。
- ② Nano 与完整版：Nano 是“只保留内核 + shell”的精简形态。本文的 rtconfig.h 里一句 #define RT_USING_DEVICE 的开与关，就是 Nano 与“带设备框架”的分水岭：

配置	控制台怎么工作	finsh 输入怎么来
不定义 RT_USING_DEVICE（纯 Nano）	rt_kprintf() 直接调用你的 rt_hw_console_output()	finsh 直接轮询你的 rt_hw_console_getchar()
定义 RT_USING_DEVICE	rt_kprintf() 找 console 设备 （你要注册一个字符设备）	走 rt_device_read()，靠 rx_indicate 唤醒

- ③ 启动流程只有三个函数（components.c）：

```
/* 你只需要实现这三个里的前两个，第三个内核自己调 */
rt_hw_board_init();           // 你要写：时钟、内存、控制台、tick (→ bsp/board.c)
rt_application_init();        // 内核提供，创建 main 线程
rt_system_scheduler_start();   // 内核提供，启动调度器，永不返回
```

- ④ tick 是你的责任：内核的“时间”全靠一个周期性中断去调 rt_tick_increase()。移植时“tick 不来”是最常见的卡点（ps 里 left tick 不动、rt_thread_mdelay() 永不返回都是这个症状）。

1.2 自动初始化表：RT-Thread 的“骨架”，也是移植里最容易踩的坑

RT-Thread 不用你手写一堆 init()；调用，而是把初始化函数指针按级别塞进链接脚本里的一个数组，启动时按地址顺序遍历调用。宏定义在 include/rtdef.h：

```

#define INIT_BOARD_EXPORT(fn)    INIT_EXPORT(fn, "1")    // 板级: 时钟、内存
#define INIT_PREV_EXPORT(fn)    INIT_EXPORT(fn, "2")
#define INIT_DEVICE_EXPORT(fn)  INIT_EXPORT(fn, "3")    // 设备: 串口、tick
#define INIT_COMPONENT_EXPORT(fn) INIT_EXPORT(fn, "4")
#define INIT_ENV_EXPORT(fn)     INIT_EXPORT(fn, "5")
#define INIT_APP_EXPORT(fn)     INIT_EXPORT(fn, "6")    // 应用: finsh_system_init 就在这一级

```

实现方式是给函数指针加一个“段名”（GCC 的 `section` 属性），例如第 1 级会落到 `.rti_fn.1` 段。链接脚本把它们按名字排序拼在一起，内核再按 `&__rt_init_rti_board_start` 到 `&__rt_init_rti_board_end` 之间的地址范围逐个调用。

移植必知的三条（这三条我在移植时全踩过，见第 4 节）：

1. **链接脚本必须 KEEP 这些段**：`rt_components_init()` 是“按地址遍历”的，链接器看不到任何引用 → 开了 `--gc-sections` 就会被当垃圾删掉。
2. **INIT_BOARD_EXPORT（第 1 级）不会自动执行**：`rt_components_init()` 只遍历 2~6 级。想让第 1 级跑，得在 `rt_hw_board_init()` 里自己调 `rt_components_board_init()`。—— 所以**驱动都挂 INIT_DEVICE_EXPORT（3 级）最省心**。
3. **段所在的内存不能被别的机制覆盖**（典型是堆），否则遍历时会跳到莫名其妙的地址。

调试“某个初始化没执行”的通用手段：把整张表摊开打印（段区间 + 每项的目标地址 + 是否落在镜像区间内）。本工程 `bsp/board.c` 的 `s31_dump_init_table()` 就是干这个的，一次开机就能定位问题。

1.3 一个“裸机工程”到底缺什么

平时用 SDK/IDE 时被自动补上的东西，移植时都变成“你必须自己给”：

缺什么	谁负责给	本工程在哪
C 运行环境（栈指针、 <code>.data</code> 搬移、 <code>.bss</code> 清零、 <code>gp</code> 寄存器）	启动汇编（正常是工具链的 <code>crt0</code> ）	<code>bsp/startup.S</code>
链接脚本（内存布局、段顺序、堆栈位置）	你	<code>bsp/linker.ld</code>
中断/异常向量	你（RISC-V 上是 <code>mtvec</code> + 一段汇编）	<code>bsp/trap_gcc.S</code> 、 <code>bsp/trap_handler.c</code>
时钟（CPU 主频）	你	<code>bsp/drv_clk.c</code> （本文只用默认 40MHz）
周期性 tick	你（一个定时器 + 一个中断）	<code>bsp/drv_systick.c</code>
控制台（ <code>printf</code> /输入）	你	<code>bsp/drv_usj.c</code>
系统调用桩（ <code>_sbrk</code> / <code>_write</code> ...）	你（链 <code>newlib</code> 时需要）	<code>bsp/syscalls_stub.c</code>
构建脚本	你（本文用逐文件编译的 PowerShell 脚本）	<code>tools/build.ps1</code>

RISC-V 上额外补一点：本文涉及这几个 CSR（都用 `csrr`/`csrw` 读写），看到它们不要慌，作用一句话就能说清：

CSR	作用	本文在哪用到
<code>mhartid</code>	当前是哪个 hart（核）	<code>startup.S</code> ：只让 <code>core0</code> 干活，其他核停车
<code>mtvec</code>	陷阱入口地址（中断/异常都跳这里）	<code>startup.S</code> ：指向 CLIC 入口
<code>mstatus</code>	全局中断使能位 <code>MIE</code>	判断“当前在不在中断里”
<code>mepc</code> / <code>mcause</code> / <code>mtval</code>	出错在哪、什么错、错的地址/值	异常打印现场
<code>mscratch</code>	一个备用寄存器，用来存“中断栈指针”	切换中断栈

另外 RISC-V 的 **gp (global pointer)** 很关键：编译器把“小静态变量”（≤8 字节）放进 `.sdata/.sbss`，用 `gp` 相对寻址访问。**没有 `crt0` 就没人设置 `gp`**，此时读写这些小变量会落到随机地址——本工程在链接脚本给出 `__global_pointer$`，在 `startup.S` 里装载它。

2. 先看清这块板怎么启动（它决定了移植方式）

移植的第一步不是写代码，而是搞清楚芯片从上电到跳进你的 `main` 之间发生了什么。ESP32-S31（乐鑫 2025 年的新芯片，RISC-V 双核 + LP 核）的情况是：

上电/复位

- └─ bootROM (芯片内部固化的固件, 改不了)
 - └─ 读 flash 0x2000 处的"二级镜像" (esptool 的 BOOTLOADER_FLASH_OFFSET = 8192)
 - └─ 按镜像头里的段表, 把每个 LOAD 段搬到它自己的 VMA (都在 0x2F000000 起的 RAM 里)
 - └─ 跳到镜像头里的 entry — 也就是我们的 _start

关键推论 (这几条直接决定了工程形态) :

1. **镜像就是"能被 bootROM 认的 ELF 转出来的 bin"** : 段全部放 RAM, 不需要自己做 flash 映射/MMU。→ 链接脚本可以非常简单。
2. **不用 IDF 的二级 bootloader, 就意味着自己扛下它干的活** : 关看门狗、初始化内存、初始化控制台、初始化时钟。少了它, **上电后最先要命的是看门狗**。
3. **RAM 窗口** : 0x2F000000 .. 0x2F07AFC0 (491456 字节) 。出处: soc.h:152 + ld.hp_mem_defs:9 ; ROM 自己的栈在 0x2F07CFB0 (在窗口之上, 不冲突) 。
4. **日志通路只有 USB-Serial/JTAG** (就是电脑上那个 COM 口) : 板子上 UART0 没接出来, ROM 自己也是用它打印的, 所以这是我们**唯一的** printf 出口。
5. **中断控制器是 CLIC** (不是老 ESP32 那种中断矩阵) , 外设中断要**经过路由矩阵**映射到 CLIC ID。
6. 这块板上还有 **SYSTIMER** : 一个固定在 XTAL 40MHz / 2.5 = 16MHz 的计数器, **与 CPU 主频无关**——它是做 tick 和"测时间"的理想时基。

建议的"移植第 0 步" : 先写一个**最小裸机固件** (不含 RT-Thread, 几行 C) , 只验证三件事: 镜像能被 bootROM 跳进来、能通过 USB-Serial/JTAG 打印、能把 .bss 清零并正常调用函数。这一步过了, 后面所有失败都能归因到"RT-Thread 配置/链接", 不会再怀疑"是不是镜像格式不对"。本工程最早就是靠这个冒烟测试打底, 后来它完成了使命就被删掉了。

3. 移植步骤 (九步, 每步都有验证方法)

结论先行: **步骤 2~4 是"能不能跑起来"**, **步骤 5~7 是"能不能看见和响应"**, **步骤 8~9 是"能不能用"**。出问题时按这个顺序回退排查, 不要跳步。

步骤 1: 目录骨架、工具链、拿到 RT-Thread 源码

目标 : 一条命令能拉到固定版本的 RT-Thread, 源码不进 git。

产出 :

```
rtt_nano_s31/
├─ bsp/          自己的板级代码（本文 90% 的活在这）
├─ app/main.c    main 线程 + msh 命令
├─ tools/        fetch_rtt.ps1（拉源码）、build.ps1（编译+烧录）
├─ rt-thread/    上游源码（稀疏检出，不进 git）
└─ prebuilt/     备份的原 IDF bootloader（哪天想恢复 IDF 启动用）
```

关键点：

- **工具链用绝对路径**，不加载 IDF 环境：`%USERPROFILE%\espressif\tools\riscv32-esp-elf\esp-15.2.0_20251204\...\riscv32-esp-elf-gcc.exe`。
- **烧录用独立的 esptool**：`python -m esptool`（5.4.0 起原生支持 esp32s31）。
- **RT-Thread 用 git 稀疏检出固定 tag**（本工程是 v5.2.2），只拿需要的东西：内核 `src/`、`klibc` `src/klibc/`、`finsh` `components/finsh/`、RISC-V 端口 `libcpu/risc-v/`、`include/`。这样“RT-Thread 是哪一版”永远可复现（`tools/fetch_rtt.ps1` 会把 commit 记进 `rt-thread.commit`）。

验证：`rt-thread/src/` 等目录存在，且 `git status` 里看不到 `rt-thread/`（已被 `.gitignore` 排除）。

步骤 2：链接脚本 `bsp/linker.ld`

目标：把“镜像 → 堆 → 中断栈 → 启动栈”在 RAM 窗口里排好，并且让三张表活下来。

段顺序（自底向上）：

```
.boot → .clic_entry(64B 对齐) → .text → .rodata
    → .rti_fn（自动初始化表）→ .fsymtab（msh 命令表）
    → .data → .sdata → .bss → .sbss → .noinit
    → 堆 → 中断栈 4KB → 启动栈 8KB（栈顶 = 窗口顶 0x2F07AFC0）
```

三张“必须 KEEP”的表，写法如下（这是整份链接脚本最重要的 10 行）：

```
.rti_fn : ALIGN(4) {
    __rti_fn_start = .;
    KEEP(*(SORT(.rti_fn*)))    /* SORT 保证 0 < 0.end < 1 < ... < 6.end 的顺序 */
    __rti_fn_end = .;
} > RAM

.fsymtab : ALIGN(4) {
    __fsymtab_start = .;
    KEEP(*(FSymTab))          /* msh 命令表 */
    __fsymtab_end = .;
} > RAM
```

还要处理两件 RISC-V 特有的事：

```
/* gp: 正常由 crt0 设置；我们 -nostartfiles 没有 crt0 */  
PROVIDE(__global_pointer$ = MIN(__sdata_start + 0x800,  
                                MAX(__sdata_end, __sbss_end) - 0x800));
```

并且用 ASSERT 把“布局错误”变成编译期错误（比运行期玄学强一百倍）：

```
ASSERT(__noinit_end <= __heap_end, "BSS/NOINIT OVERLAPS HEAP")  
ASSERT(__heap_end >= __fsymtab_end, "HEAP OVERLAPS INIT/FINSH TABLES")  
ASSERT((s31_cllic_entry & 0x3F) == 0, "s31_cllic_entry MUST be 64-byte aligned")
```

验证：riscv32-esp-elf-nm -n build/app.elf 看 __rti_fn_start/__rti_fn_end、__fsymtab_start/__fsymtab_end、__heap_start/__heap_end 是否都在 RAM 窗口内、且顺序正确。

步骤 3：启动汇编 bsp/startup.S

目标：接管 bootROM 交过来的控制权，把 C 运行环境铺好。

要按顺序做这几件事：

```

_start:
    csrr    t0, mhartid          /* ① 只让 core0 干活，其它核停车 */
    bnez    t0, .Lpark

    /* ② 关看门狗 —— 见下面"四个看门狗"的说明 */
    li      t1, 0x50D83AA1
    li      t0, 0x20801000      /* RTC_WDT */
    sw      t1, 0x18(t0); sw zero, 0x00(t0); sw zero, 0x18(t0)
    li      t0, 0x20580000      /* TIMG0 MWDT */
    sw      t1, 0x64(t0); sw zero, 0x48(t0); sw zero, 0x64(t0)
    /* ...TIMG1 同理：再打开"超级看门狗"的自动喂狗（见第 4 节） */

    la      sp, __stack_top     /* ③ 装启动栈 */
    .option norelax
    la      gp, __global_pointer$ /* ④ 装 gp（小数据寻址全靠它） */
    .option relax

    la      t0, s31_cltic_entry /* ⑤ mtvec = CLIC 入口 | 3 */
    ori     t0, t0, 3
    csrw    mtvec, t0

    la      t0, __bss_start     /* ⑥ 清 .bss（+ .sbss） */
    la      t1, __bss_end
1: bgeu     t0, t1, 2f
    sw      zero, 0(t0); addi t0, t0, 4; j 1b
2: call     entry              /* ⑦ 进 RT-Thread 的 entry() */

```

关于看门狗（这块板上有四个！）：不关就看不停复位，而且**症状各不相同**。本工程实测：RTC_WDT / TIMG0 MWDT / TIMG1 MWDT 三个都要在第一条指令附近关掉；第四个“超级看门狗（SWD）”**不能只关，要喂**——它盯的是 RTC_WDT 的喂狗信号，你把 RTC_WDT 关了它就没人喂，于是**每 3.4 秒整片复位一次**（这个坑见第 4 节，很难查）。

验证：镜像能起来并打印（下一步做完就能看到），或者用调试器 `reg pc` 确认 PC 落在 `.boot/.text`。

步骤 4: bsp/rtconfig.h (Nano 配置)

目标：告诉内核“我要什么”。

最小可用集合（完整见文件）：

```
#define RT_NAME_MAX 12
#define RT_THREAD_PRIORITY_MAX 32
#define RT_TICK_PER_SECOND 1000 /* tick = 1ms */
#define RT_USING_SMALL_MEM /* 小内存管理 */
#define RT_USING_SMALL_MEM_AS_HEAP
#define RT_USING_HEAP
#define RT_USING_CONSOLE /* rt_kprintf */
#define RT_CONSOLEBUF_SIZE 128
#define ARCH_RISCV
#define ARCH_CPU_32BIT
#define RT_USING_COMPONENTS_INIT /* 自动初始化表 */
#define RT_USING_USER_MAIN /* main 线程 */
#define RT_USING_FINSH /* msh */
#define FINSH_USING_MSH
#define FINSH_USING_SYMTAB /* msh 命令表 */
#define RT_BACKTRACE_LEVEL_MAX_NR 32 /* kservice.c 里无 guard 引用, 必须给 */
```

三个容易漏的点：

- 1. RT_USING_USER_MAIN + RT_USING_COMPONENTS_INIT 是 msh 能自动起来的前提（finsh_system_init() 靠 INIT_APP_EXPORT 注册，在 main 线程里被调用）。
- 2. 不要定义 RT_USING_DEVICE（这一步是纯 Nano）：控制台直接走你的 rt_hw_console_output()/rt_hw_console_getchar()，省掉设备框架这一层。等以后要接 SPI/I2C 这类设备驱动时再打开（那时要写一个字符设备，见第 6 节末尾）。
- 3. 内核源码必须带 -D__RT_KERNEL_SOURCE__ 编译，否则看不到内部调度接口（rtsched.h:112 才暴露）；而 finsh/应用不能带这个宏 —— 所以构建脚本要按目录区分编译参数。

验证：这一步没法单独验证，跟着步骤 5~9 一起看结果。

步骤 5：控制台 bsp/drv_usj.c（能打印，就成功了一半）

目标：让 rt_kprintf() 真的能从 COM 口出来、并能收到字符。

硬件侧只有两个寄存器（USB-Serial/JTAG）：

寄存器	作用
EP1	写=往 TX FIFO 塞一个字节；读=从 RX FIFO 取一个字节
EP1_CONF	bit0 wr_done（把 FIFO 里的内容交给主机）、bit1 TX FIFO 有空位、bit2 RX 有数据

两个必须实现的钩子：

```
void rt_hw_console_output(const char *str);          /* rt_kprintf 的最终出口 */
signed char rt_hw_console_getchar(void);           /* 纯 Nano 下 finsh 的输入来源 */
```

为什么发送要用“环形缓冲 + 定时泵”，而不是直接写 FIFO？因为 TX FIFO 只有 64 字节，而开机横幅有 2KB；更坑的是 打开串口这个动作本身会让芯片复位（USB-JTAG 的 DTR/RTS 就是复位线），复位后主机还没开始读，直接往 FIFO 灌就必然丢开头几行（实测丢过 3 行）。所以：rt_kprintf → 2KB 环形缓冲 → 由“本次输出结束”和“每 1ms 的 tick 中断”两个时机灌进 FIFO。

一个反直觉的细节（后来踩到，写在这里当预防针）：wr_done 每次都要写，哪怕一个字节都没写。原因：装满 64 字节时硬件会自动提交这个整包，而主机会把整包当成“未结束的 USB 事务”，必须再补一个“零长度包”（= 再写一次 wr_done）才会真正交给 CDC 读端。少写这一下 → 主机读请求永远完不成 → 串口一个字节都收不到。

验证：

```
python tools/read_port.py COM43 5 # 应该看到开机打印
```

步骤 6：时钟与 tick —— bsp/drv_systick.c + CLIC 路由

目标：让内核的“时间”走起来。这是移植里最关键、最容易卡的一步。

硬件选择：用 SYSTIMER。它的时钟固定是 XTAL 40MHz / 2.5 = 16MHz，与 CPU 主频无关（以后 CPU 超频到 320MHz，tick 不受影响，这也是我用它当“时间基准”的原因）。

配置顺序必须照 IDF（顺序错了症状很怪：只中断一次、或干脆不来）：

```
/* ① 开时钟门控 ② 开 unit ③ 计数器清零 ④ 配周期与目标
   ⑤ COMP0_LOAD ⑥ 清中断标志 ⑦ 最后才 arm */
S31_REG32(SYSTIMER_CTRL0) |= APB_CLK_EN | CLK_EN;
S31_REG32(SYSTIMER_CONF)  |= CLK_EN | UNIT0_WORK_EN;
S31_REG32(SYSTIMER_TARGET0_CONF) = PERIOD_MODE | (16000 - 1); /* 16MHz / 16000 = 1ms
                                                                    */
S31_REG32(SYSTIMER_COMP0_LOAD) = 1;
/* ... */
```

中断侧要过三道关（RISC-V + CLIC 特有）：

1. 路由：外设中断源（这里是 33 = ETS_SYSTIMER_TARGET0）要写进路由矩阵 0x20585000 + 4*33 = CLIC ID。我们选 CLIC ID 16。
2. CLIC 配置：ATTR=0（电平触发）、CTL=优先级、IE=1。优先级必须 ≥1：CLIC 每路优先级复位默认 0x1F，而阈值 mintthresh 也是 0x1F，阈值恰好屏蔽 level 0 → 写 0 就永远不触发。
3. mtvec 必须 64 字节对齐：CLIC 模式下 mtvec[31:6] 是处理函数基址，且模式位 mtvec[1:0]=3。工具链的 SW_handler 落在段内偏移 0x2（汇编器给 .align 留的填充），所以本工程自己写了一个 64B 对齐的跳板（bsp/trap_gcc.S：j SW_handler）。

中断服务函数只要一件事：

```
static void s31_tick_isr(int irq, void *param)
{
    S31_REG32(SYSTIMER_INT_CLR) = T0_INT;    /* 电平触发必须清，否则反复进中断 */
    s31_usj_tx_pump();                        /* 顺手把控制台缓冲灌出去 */
    rt_tick_increase();                       /* 内核的"心跳" */
}
```

挂载时机：用 `INIT_DEVICE_EXPORT`（3 级），**不能放** `INIT_BOARD_EXPORT`，也不能在 `rt_hw_board_init()` 里开 tick —— 那时内核的定时器链表还没初始化，第一个 tick 就会去遍历野指针。

验证（做完步骤 5+6 就能看到）：

```
msh >ps
thread pri status ...
tidle0 31 ready ...
msh >list timer
```

left tick 在变、`rt_thread_mdelay()` 能返回，就说明 tick 通了。

步骤 7：陷阱入口与异常现场 —— `bsp/trap_gcc.S` + `bsp/trap_handler.c`

目标：中断能分发、出错时能看清现场。

两层结构：

- `trap_gcc.S`：64B 对齐跳板 + 保存/恢复上下文（本工程用的是 `libcpu` 现成的 `rt_hw_do_after_save_above`，它会把上下文压栈、切到中断栈（`mscratch`）、再调 C 函数）。
- `trap_handler.c`：C 侧分发。

分发的一个硬限制：RT-Thread 的中断表是按 `mcause & 0x1F` 取号的（32 项），而 CLIC 的外部中断 ID 是 16~47 → **实际只能用 16~31**。

异常分支一定要打这三个数并停机：

```
rt_kprintf("mcause=%08x mepc=%08x mtval=%08x mstatus=%08x\n",
           mcause, mepc, mtval, mstatus);
while (1) { }    /* 停机：不停机的话故障指令会反复执行 → 刷屏，什么也看不出来 */
```

（RT-Thread 自带的默认实现只打寄存器快照、不打这三个数，而且打完返回 → 8 秒能刷 2.3MB 日志。）

再加一条保险（后来踩到才加的）：**未注册的中断不要只打一行就返回** —— 外设的中断标志没人清，电平触发下会立刻再进来，形成死循环刷屏、shell 完全没法用。正解：打几行日志后把这一路 **CLIC IE 关掉**，

让系统活下来。

验证：故意触发一次异常（比如访问非法地址），看到上面那行打印并按预期停机 → 说明异常通路是通的。

步骤 8：构建脚本 `tools/build.ps1`（编译 → 链接 → 生成镜像 → 烧录）

目标：一条命令产出可烧的镜像并烧进去。

编译参数上的四个要点：

```
# ① 逐文件编译，按目录区分是否带 __RT_KERNEL_SOURCE__  
#     （内核 src/ 要带；drivers/finsh/app 不能带）  
# ② 架构：rv32imafc（含原子/浮点/压缩指令）+ zicsr/zifencei  
-march=rv32imafc_zicsr_zifencei -mabi=ilp32f  
# ③ -nostartfiles 而不是 -nostdlib：  
#     启动是我们自己的 startup.S，但内核要用 memcpy/memset/strlen 和 __udivdi3(libgcc)  
-nostartfiles  
# ④ 段裁剪 + 需要的头文件依赖跟踪  
-ffunction-sections -fdata-sections → 链接时 -WL,--gc-sections
```

必须配套的两件事：

- 链接脚本里 `KEEP(*(SORT(.rti_fn*))) / KEEP(*(FSymTab))`（否则 `--gc-sections` 会把表删掉）。
- 一个 `bsp/syscalls_stub.c`：链了 `newlib` 就会有 `_sbrk/_write/_read/_close/_fstat/...` 的引用，给它们空实现即可（我们不实现动态内存，堆是 RT-Thread 自己的）。

生成镜像与烧录：

```
# ELF → 可烧的 bin (DIO 模式、80MHz、16MB flash)  
python -m esptool --chip esp32s31 elf2image -fm dio -ff 80m -fs 16MB -o app.bin app.elf  
  
# 烧到 flash 0x2000 (bootROM 的二级镜像位置)  
python -m esptool --chip esp32s31 -p COM43 -b 460800 --no-stub \  
    --before default-reset --after hard-reset \  
    write-flash -fm dio -ff 80m -fs 16MB 0x2000 app.bin
```

注意：0x2000 是 IDF 二级 bootloader 的位置，烧进去等于覆盖它。留一份 `prebuilt/bootloader.bin` 备份，随时可以烧回去恢复 IDF 启动。

验证：esptool 报 `Hash of data verified.`；开机能看到 ROM 的横幅 + 你的镜像被加载的日志（`load:0x2f000000,... / entry 0x2f000000`）。

步骤 9：让 msh 跑起来（最后一步，其实不用写代码）

目标：串口里出现 `msh >` 并能执行命令。

它为什么“自己就起来了”：finsh 在 `shell.c:1019` 用 `INIT_APP_EXPORT` 注册了 `finsh_system_init()`；而 main 线程会调用 `rt_components_init()` → 遍历 2~6 级表 → 调到它 → 创建 `tshell` 线程 → 打印提示符。所以只要前面 8 步对了，msh 不需要你写一行代码。

验证清单（都通过就算移植成功）：

```
msh >help          # 命令列表
msh >ps             # 线程列表（tshell/tidle0/timer/main），left tick 在变
msh >free           # 堆总量/已用（本板约 425 KB）
msh >version        # RT-Thread 版本
msh >list device    # 设备列表（纯 Nano 时为空，正常）
```

加一条自己的命令，确认应用层也能跑：

```
static void hello(int argc, char **argv) { rt_kprintf("hello, %s\n", "S31"); }
MSH_CMD_EXPORT(hello, say hello);
```

小坑：MSH_CMD_EXPORT(fn, desc) 注册的命令名就是函数名；想用别的名字要写 MSH_CMD_EXPORT_ALIAS(fn, 命令名, desc)。

到这里，“RT-Thread 移植”这件事就完成了：内核在跑、tick 在走、中断能进、msh 能用。

4. 三个必踩的坑（现象 → 根因 → 修法）

这三个坑我在移植时全踩了，而且它们的症状都很有迷惑性，所以单独列出来。

坑 1：所有 `INIT_*_EXPORT` 的函数一个都不执行，但系统“看上去启动了”

- **现象**：版本号打印了、`main()` 也进了，但 tick 不走、msh 不起来、什么都没初始化。
- **根因**：链接时开了 `--gc-sections`，而 `rt_components_init()` 是按地址区间遍历调用这些函数指针的——链接器看不到“有人引用”，于是把 `.rti_fn.*` 整段当垃圾删掉。
- **修法**：链接脚本里 `KEEP(*(SORT(.rti_fn*)))`。SORT 保证段的先后顺序（`0 < 0.end < 1 < ...`）。

坑 2：堆把“表”写花了 → 遍历初始化表时跳到 `0x8c`

- **现象：** `instruction access fault`，`mepc` 指向 `0x8c`。
- **根因：** 两张表（`.rti_fn`、`.fsymtab`）最初放在 `.sbss` 之后，而堆起点 `__bss_end` 只算到 `.sbss` 末尾 → `rt_system_heap_init()` 把表当空闲内存写花了。
- **修法：** 凡是有符号指向的区间，都必须落在 `__heap_start` 之前；再加一条 `ASSERT(__heap_end >= __fsymtab_end)` 让它变成编译期错误。

坑 3： `INIT_BOARD_EXPORT`（1 级）永远不执行

- **现象：** 把驱动挂 1 级 → 怎么都不工作（例如 `pin` 命令 `load access fault`，`mtval=0x18`，因为 `ops` 是 `NULL`）。
- **根因：** `rt_components_init()` 只遍历 2~6 级；1 级要 BSP 自己在 `rt_hw_board_init()` 里调 `rt_components_board_init()`。
- **修法：** 驱动统一挂 `INIT_DEVICE_EXPORT`（3 级）；而且不要在 `rt_hw_board_init()` 里开 `tick`（那时内核定时器链表还没初始化）。

5. 步骤 → 文件 → 验证 总表

步骤	产出文件	干什么	怎么验证
0	(临时) 裸机冒烟固件	证明镜像格式/段布局/ROM 打印都对	打印一行 hello, .bss 清零正常
1	tools/fetch_rtt.ps1、目录骨架	固定版本拉源码	rt-thread/src 存在、不进 git
2	bsp/linker.ld	段布局 + 三张表 KEEP + 堆栈 + ASSERT	nm -n 看符号区间与顺序
3	bsp/startup.S	关四个看门狗、栈、gp、mtvec、清 bss、进 entry()	PC 落到 .text, 后面能打印
4	bsp/rtconfig.h	Nano 配置 (无设备框架)	编译通过、体积合理
5	bsp/drv_usj.c	控制台 (2KB 环形缓冲 + EP1/EP1_CONF)	read_port.py 看到开机打印
6	bsp/drv_systick.c	SYSTIMER 1ms tick + CLIC 路由/优先级	ps 里 left tick 在变
7	bsp/trap_gcc.S、bsp/trap_handler.c	中断分发 + 异常现场打印	故意触发异常能看到 mcause/mepc/mtval
8	tools/build.ps1、bsp/syscalls_stub.c	编译/链接/生成镜像/烧 0x2000	Hash of data verified + ROM 加载日志
9	app/main.c (可空)	msh 由 finsh 自动起来	msh > 下敲 help/ps/free/version

一条命令复现整个流程：

```
pwsh -File tools\fetch_rtt.ps1 # ① 拉 RT-Thread (v5.2.2)
pwsh -File tools\build.ps1 # ② 编译 + 烧录 + 抓开机日志
pwsh -File tools\msh.py COM43 "help|ps|free|version" 3 # ③ 敲命令看回显
```

6. 往下走：设备框架（你已经熟悉，这里只做衔接）

本文停在“纯 Nano + msh”。下一步通常是接入设备驱动，衔接点只有一个开关：在 rtconfig.h 里加上

```
#define RT_USING_DEVICE
#define RT_USING_PIN /* 以及 SPI / I2C 等 */
```

打开之后控制台的行为会变：

- `rt_kprintf()` 去找 console 设备（没有的话回退到你的 `rt_hw_console_output()`）；
- `finsh` 改用 `rt_device_read()` + `rx_indicate` 唤醒。

所以此时你要做的是：把原来的轮询控制台**包装成一个字符设备**（本工程 `bsp/drv_usj_dev.c`）：实现 `open/close/read/write`，在 USB 的 RX 中断里把数据搬进环形缓冲、调用 `rx_indicate()` 唤醒 `finsh`，最后 `rt_console_set_device("usj")`。

顺序上的一个硬要求（我在设备驱动阶段才踩到，但根因在移植阶段）：中断的“装处理函数”和“使能 CLIC IE”**顺序不能反**——先使能 IE、后装 handler，一旦此时外设中断已经是 pending，CPU 会带着 NULL handler 进中断，而那一路再也装不上，表现就是死循环刷屏。

附录 A：本文用到的关键地址与出处

名称	地址/值	出处
二级镜像偏移	0x2000	esptool esp32s31.BOOTLOADER_FLASH_OFFSET
RAM 窗口	0x2F000000 .. 0x2F07AFC0	soc.h:152、ld.hp_mem_defs:9
USB-Serial/JTAG	0x20391000（EP1 = +0x00、EP1_CONF = +0x04）	register/soc/reg_base.h
SYSTIMER	0x20399000	同上
CLIC 路由矩阵	0x20585000 + 4*source	hal/interrupt_clic_ll.h
CLIC 阈值 CSR	minthresh = 0x347	RISC-V CLIC 规范
中断源 33 / 2	SYSTIMER_TARGET0 / USB_SERIAL_JTAG	soc/interrupts.h
RTC_WDT / TIMG0 / TIMG1	0x20801000 / 0x20580000 / 0x20581000	register/soc
看门狗解锁 key	0x50D83AA1	hal/wdt_hal.h

附录 B：术语速查

术语	一句话解释
BSP	板级支持包：把内核“接到”具体硬件上的那一层代码（本工程的 <code>bsp/</code> ）
tick	内核的时间单位（这里 1ms），由你的定时器中断驱动 <code>rt_tick_increase()</code>
自动初始化	用段名 + 链接脚本把 <code>INIT*_EXPORT</code> 的函数指针排成表，启动时按级遍历调用
klibc	RT-Thread 自带的轻量 C 库（ <code>rt_kprintf</code> / <code>rt_memset</code> / <code>rt_vsnprintf</code> 等）
finsh / msh	RT-Thread 的 shell；msh 是它的“命令行模式”
CLIC	RISC-V 的核内中断控制器（替代老的中断矩阵），支持每路独立优先级/电平触发
CSR	RISC-V 的控制状态寄存器（ <code>mtvec</code> / <code>mstatus</code> / <code>mcause</code> ...），用 <code>csrr</code> / <code>csrw</code> 访问
gp	RISC-V 的全局指针寄存器，指向小数据区，用于 gp 相对寻址
gc-sections	链接器裁剪“没人引用”的段；会顺手删掉自动初始化表，必须 KEEP